Estudo Comparativo entre diferentes Algoritmos de Ordenação

Pedro Henrique Rodrigues Vargas
Universidade Federal do Espírito Santo - UFES

21 de janeiro de 2026

Resumo

Com o avanço da tecnologia, a eficiência dos algoritmos de ordenação tornou-se crucial para o desempenho de sistemas computacionais. Este artigo apresenta um estudo comparativo entre diversos algoritmos de ordenação, analisando sua complexidade temporal e espacial em diferentes cenários de uso. Os resultados indicam que a escolha do algoritmo adequado pode impactar significativamente a performance de aplicações que lidam com grandes volumes de dados. Por isso, avaliamos 14 diferentes algoritmos de ordenação, do famoso algoritmo de bolha, *BubbleSort*, ao *BucketSort*, realizando diferentes testes, em diferentes cenários pensados para encontrarmos os melhores resultados possíveis. **Palavras-chave:** Algoritmos de Ordenação. Eficiência. Complexidade.

1 Introdução

Neste artigo, exploramos 14 algoritmos de ordenação, incluindo ***BubbleSort***, ***InsertionSort***, ***BinarySort***, ***TernarySort***, ***ShellSort***, ***SelectionSort***, ***HeapSort***, ***QuickSort***, ***MergeSort***, ***RadixSort*** e ***BucketSort*** . Cada algoritmo foi implementado utilizando a linguagem C e para cada algoritmo, foram realizados testes em diferentes cenários: listas já ordenadas, listas inversamente ordenadas e listas com elementos aleatórios, com tamanhos variados. A análise de desempenho foi baseada no tempo de execução e complexidade de cada algoritmo.

Em um primeiro momento, iniciamos avaliando os algoritmos utilizando vetores pequenos, com 10000 elementos, organizados da seguinte forma: aleatórios, crescentes e decrescentes. Então, partimos para vetores maiores, com 100000 de elementos, e 500000 de elementos, sempre mantendo a mesma organização.

Vale destacar que o volume da dados que algumas empresas lidam atualmente é imenso, em casos passando facilmente da casa dos bilhões de registros. Porém nesse teste, optamos por não utilizar vetores maiores que 500000 elementos, pois como será apresentado mais adiante, o tempo de execução de alguns algoritmos com complexidade quadrática se torna irrazoavelmente enfático em relação aos resultados de algoritmos mais eficientes à medida que adicionamos mais elementos. Portanto, o tamanho de 500000 já é suficiente para demonstrar a ineficiência desses algoritmos em cenários mais realistas. A seguir, vamos apresentar os códigos de cada algoritmo.

2 Algoritmos

Nesta seção estão todos os 14 algoritmos de ordenação que foram utilizados no estudo comparativo. Para não prejudicar a legibilidade do artigo, as implementações foram exibidas no final do artigo, e apenas uma breve descrição de cada algoritmo é apresentada aqui.

Algoritmos simples como *BubbleSort* e *InsertionSort* são apresentados primeiro, seguidos por algoritmos mais complexos como *QuickSort* e *MergeSort*.

2.1 BubbleSort

O BubbleSort é um algoritmo de ordenação simples que repetidamente percorre a lista, compara elementos adjacentes e os troca se estiverem na ordem errada. Essa passagem pela lista é repetida até que a lista esteja ordenada.

2.2 BubbleSort com Parada

Uma otimização do BubbleSort é adicionar uma verificação para ver se alguma troca foi feita durante uma passagem pela lista. Se nenhuma troca for feita, a lista já está ordenada e o algoritmo pode parar mais cedo.

2.3 InsertionSort

O InsertionSort é um algoritmo de ordenação que constrói a lista ordenada um elemento de cada vez. Ele pega cada elemento da lista de entrada e o insere na posição correta na lista ordenada.

2.4 BinarySort

A Inserção Binária é uma variação do InsertionSort que utiliza busca binária para encontrar a posição correta de inserção de cada elemento, reduzindo o número de comparações necessárias.

2.5 TernarySort

A Inserção Ternária é uma variação do InsertionSort que utiliza busca ternária para encontrar a posição correta de inserção de cada elemento, dividindo a lista em três partes para reduzir ainda mais o número de comparações necessárias.

2.6 ShellSort

O ShellSort é um algoritmo de ordenação que generaliza o InsertionSort para permitir a troca de elementos distantes. Ele começa comparando elementos que estão longe uns dos outros e gradualmente reduz a distância entre os elementos a serem comparados.

2.7 SelectionSort

O SelectionSort é um algoritmo de ordenação que divide a lista em duas partes: a parte ordenada e a parte não ordenada. Ele repetidamente seleciona o menor (ou maior) elemento da parte não ordenada e o move para o final da parte ordenada.

2.8 HeapSort

O heapSort é um algoritmo de ordenação baseado na estrutura de dados heap. Ele constrói um heap a partir dos dados de entrada e, em seguida, extrai o maior (ou menor) elemento do heap repetidamente para construir a lista ordenada. Ele faz isso em duas fases: primeiro, constrói um heap máximo a partir do array de entrada; depois, remove o maior elemento do heap (a raiz) e o coloca no final do array, reduzindo o tamanho do heap em um. Esse processo é repetido até que todos os elementos estejam ordenados.

2.9 Quicksort

O QuickSort é um algoritmo de ordenação eficiente que utiliza a técnica de divisão e conquista. Ele seleciona um elemento como pivô e particiona a lista em duas sublistas: uma com elementos menores que o pivô e outra com elementos maiores que o pivô. Em seguida, ele ordena recursivamente as sublistas. Aqui, temos três variações do QuickSort, cada uma utilizando uma estratégia diferente para escolher o pivô: no centro, no fim e na mediana do vetor.

2.10 MergeSort

O MergeSort é um algoritmo de ordenação eficiente que também utiliza a técnica de divisão e conquista. Ele divide a lista em duas metades, ordena cada metade recursivamente e, em seguida, mescla as duas metades ordenadas para produzir a lista final ordenada.

2.11 RadixSort

O RadixSort é um algoritmo de ordenação linear não comparativo que ordena os números inteiros processando individualmente cada dígito. Ele começa pelo dígito menos significativo e avança para o dígito mais significativo, utilizando um algoritmo de ordenação estável (como o CountingSort) para ordenar os números com base em cada dígito.

2.12 BucketSort

O BucketSort é um algoritmo de ordenação que distribui os elementos de entrada em um número finito de "baldes"(ou "buckets"). Cada balde é então ordenado individualmente, geralmente usando outro algoritmo de ordenação, e os baldes são concatenados para formar a lista ordenada final.

3 Resultados

3.1 Vetores com 10000 elementos

Aqui estão os resultados dos testes realizados com vetores de 10000 elementos.

Tabela 1: Resultados para N = 10000

	Aleatório			Crescente			Decrescente		
Algoritmo	Comp.	Troca	Tempo (s)	Comp.	Troca	Tempo (s)	Comp.	Troca	Tempo (s)
Bolha	49995000	24952888	0,1239759	49995000	0	0,0659426	49995000	49995000	0,1451197
Bolha com Parada	49981281	24952888	0,1130962	9999	0	0,0000119	49995000	49995000	0,1392079
Seleção Direta	49995000	9992	0,0621527	49995000	0	0,0606472	49995000	5000	0,062966
Ordena Binária	118964	24952888	0,0261411	113631	0	0,0002799	123617	49995000	0,0508815
Ordena Ternária	127826	24953471	0,0277247	150486	0	0,0002874	75243	49995000	0,0523209
Ordena Direta	24962883	24952888	0,0430634	9999	0	0,0000278	49995000	49995000	0,0889183
ShellSort	241227	170323	0,0009789	75243	0	0,0001546	120190	53704	0,0002533
HeapSort	235476	124263	0,0009548	244460	131956	0,0007409	226682	116696	0,000716
MergeSort	120494	267232	0,0018694	69008	267232	0,0011705	64608	267232	0,0010473
Quicksort Centro	178399	35092	0,000842	143615	9999	0,0002685	143614	14998	0,0002795
Quicksort Fim	153440	32287	0,0014703	49995000	9999	0,0411658	50000000	9999	0,0430158
Quicksort Mediana	189379	41162	0,0008572	173612	9999	0,0002657	173611	15000	0,0003698
RadixSort	9999	100000	0,0003493	9999	80000	0,000486	9999	100000	0,0004846
BucketSort	9999	10000	0,0001736	9999	10000	0,0000521	9999	10000	0,0000704

Podemos perceber alguns detalhes interessantes nos resultados apresentados na tabela acima.

3.2 Vetores com 100000 elementos

Aqui estão os resultados dos testes realizados com vetores de 100000 elementos.

Tabela 2: Resultados para N = 100000

	Aleatório			Crescente			Decrescente		
Algoritmo	Comp.	Troca	Tempo (s)	Comp.	Troca	Tempo (s)	Comp.	Troca	Tempo (s)
Bolha	4999950000	2505816708	19,1008208	4999950000	0	6,6701836	4999950000	4999950000	13,7940892
Bolha com Parada	4999368699	2505816708	18,7658825	99999	0	0,0001249	4999950000	4999950000	14,0656675
Seleção Direta	4999950000	99985	6,039928	4999950000	0	5,9517727	4999950000	50000	6,2720383
Ordena Binária	1522852	2505816708	2,6248363	1468946	0	0,0038788	1568929	4999950000	5,2030733
Ordena Ternária	1653240	2505883687	2,6118072	1934292	0	0,0037757	967146	4999950000	5,0971373
Ordena Direta	2505916692	2505816708	4,4281424	99999	0	0,0001784	4999950000	4999950000	8,8602179
ShellSort	3871671	2946018	0,0145472	967146	0	0,0018053	1533494	619654	0,0032986
HeapSort	3019599	1574963	0,0120167	3112517	1650854	0,0084957	2926640	1497434	0,0085386
MergeSort	1536316	3337856	0,01887	853904	3337856	0,010753	815024	3337856	0,0104155
Quicksort Centro	2226873	459428	0,0092934	1768927	99999	0,0021882	1768926	149998	0,0023402
Quicksort Fim	2008628	429734	0,0083882	4999950000	99999	4,4302302	5000000000	99999	4,4770111
Quicksort Mediana	2272723	499545	0,0100095	2068924	99999	0,0030056	2068923	150000	0,0030327
RadixSort	99999	1000000	0,0035062	99999	1000000	0,003388	99999	1200000	0,0040687
BucketSort	99999	100000	0,0006604	99999	100000	0,0004004	99999	100000	0,0004745

3.3 Vetores com 500000 elementos

Aqui estão os resultados dos testes realizados com vetores de 500000 elementos.

Tabela 3: Resultados para $N = 500.000$

	Aleatório			Crescente			Decrescente		
Algoritmo	**Comp.**	**Troca**	**Tempo (s)**	**Comp.**	**Troca**	**Tempo (s)**	**Comp.**	**Troca**	**Tempo (s)**
Bolha	1.25×10^{11}	6.25×10^{10}	485,35	1.25×10^{11}	0	162,75	1.25×10^{11}	1.25×10^{11}	343,42
Seleção Direta	1.25×10^{11}	499.970	147,50	1.25×10^{11}	0	145,09	1.25×10^{11}	250.000	155,78
Ordena Binária	8.774.032	6.25×10^{10}	64,37	8.475.732	0	0,022	8.975.713	1.25×10^{11}	128,87
Ordena Ternária	9.642.168	6.25×10^{10}	64,65	11.202.852	0	0,023	5.601.426	1.25×10^{11}	128,68
Ordena Direta	6.25×10^{10}	6.25×10^{10}	110,55	499.999	0	0,0009	1.25×10^{11}	1.25×10^{11}	221,92
ShellSort	26.885.089	21.521.058	0,093	5.601.426	0	0,010	8.622.570	3.451.804	0,019
HeapSort	17.397.767	9.024.461	0,070	17.837.785	9.355.700	0,048	16.977.997	8.668.450	0,047
MergeSort	8.837.086	18.951.424	0,092	4.783.216	18.951.424	0,054	4.692.496	18.951.424	0,054
QuickSort (Centro)	12.729.696	2.805.605	0,047	9.975.711	499.999	0,012	9.975.710	749.998	0,012
QuickSort (Fim)	11.055.076	2.643.687	0,044	1.25×10^{11}	499.999	**111,88**	1.25×10^{11}	499.999	**115,19**
QuickSort (Mediana)	12.531.711	2.942.659	0,049	11.475.708	499.999	0,014	11.475.707	750.000	0,015
RadixSort	499.999	5.000.000	0,017	499.999	6.000.000	0,021	499.999	6.000.000	0,021
BucketSort	499.999	500.000	**0,0019**	499.999	500.000	**0,0023**	499.999	500.000	**0,0025**

3.4 Análise dos Resultados

3.4.1 Algoritmos de Complexidade Quadrática

Bolha

O algoritmo de bolha apresentou o pior tempo de execução em todos os cenários testados, especialmente em vetores maiores. Isso se deve à sua complexidade de tempo $O(n^2)$, que o torna ineficiente para grandes conjuntos de dados. Mesmo com a otimização de parada antecipada, o desempenho não melhorou significativamente para vetores grandes, porém garantiu que no melhor caso (vetores já ordenados) o tempo de execução fosse praticamente desprezível.

Seleção Direta

O SelectionSort também mostrou tempos de execução elevados, embora ligeiramente melhores que o BubbleSort. Ele se manteve estável nos piores e melhores casos. Em relação ao números de trocas, se manteve campeão entre os algoritmos quadráticos, realizando poucas substituições em todos os cenários.

Inserção Direta

Se apresentou bem instável, tendo tempos extremamente díspares no melhor e pior caso. Isso se deve ao fato de que no melhor caso (vetores já ordenados), o algoritmo realiza poucas comparações e trocas, resultando em um tempo de execução muito baixo. No entanto, no pior caso (vetores inversamente ordenados), o número de comparações e trocas aumenta significativamente, levando a um tempo de execução muito maior.

Binária e Ternária

Ambos os algoritmos apresentaram-se na média, os mais eficientes entre os algoritmos de complexidade quadrática. A Inserção Binária se saiu um pouco melhor que a Ternária, mas ambos tiveram desempenhos semelhantes. O gargalo foi o número de trocas, pois ambos os algoritmos ainda dependem do deslocamento dos elementos para inserir o novo elemento na posição correta.

3.4.2 Algoritmos de complexidade Linearítmica

ShellSort

Para o primeiro algoritmo eficiente analisado, o Shellsort apresentou resultados interessantes: números de trocas, comparações e tempos de execução relativamente baixos em todos os cenários, bem menores do que os algoritmos quadráticos. isso se deve ao fato de que o ShellSort reduz significativamente o número de deslocamentos necessários para ordenar a lista, especialmente em listas maiores, pois utiliza uma sequência de intervalos decrescentes para comparar elementos distantes.

HeapSort

O HeapSport apresentou um desempenho consistente em todos os cenários testados. Seus números de comparações e trocas foram moderados, e os tempos de execução foram razoáveis, embora não tão baixos quanto os algoritmos mais eficientes como o MergeSort e o QuickSort.

MergeSort

O MergeSort demonstrou um desempenho sólido em todos os cenários, com números de comparações e trocas relativamente baixos. Se destacou por sua estabilidade e eficiência, especialmente em listas maiores, onde seu tempo de execução foi significativamente menor do que os algoritmos quadráticos, porém ainda maior que os tempos do quicksort.

QuickSort

Em relação às variações do quicksort, a escolha do pivô teve um impacto significativo no desempenho do algoritmo. O QuickSort com pivô no centro e o QuickSort com mediana apresentaram desempenhos semelhantes, ambos superando o QuickSort com pivô no fim. Isso se deve ao fato de que escolher o pivô no centro ou na mediana tende a resultar em partições mais equilibradas, o que é crucial para a eficiência do QuickSort. Porém, o pivô no fim pode levar a partições desbalanceadas, especialmente em listas já ordenadas ou quase ordenadas, resultando em um desempenho degradado próximo ao pior caso $O(n^2)$.

3.4.3 Algoritmos Lineares

RadixSort

O RadixSort apresentou uma diferença enorme em comparação com os algoritmos anteriores. Ele demonstrou números de comparações muito baixos, uma vez que sua complexidade é linear em relação ao número de elementos e ao número de dígitos nos elementos.

BucketSort

O BucketSort foi o algoritmo mais eficiente em todos os cenários testados. Ele apresentou os menores números de comparações, trocas e tempos de execução. Isso se deve ao fato de que o BucketSort distribui os elementos em baldes e ordena cada balde individualmente, O que elimina a necessidade de busca e deslocamento extensivo de elementos.

4 Conclusão

A análise comparativa dos algoritmos de ordenação revelou diferenças significativas em termos de eficiência e desempenho.

Os algoritmos de complexidade quadrática, como o BubbleSort e o SelectionSort, mostraram-se inadequados para grandes conjuntos de dados devido aos seus altos tempos

de execução. Algoritmos quadráticos, como o BubbleSort, na verdade, não são utilizados em aplicações práticas devido à sua ineficiência, sendo mais relevantes apenas para fins educacionais. Como constatou-se nas tabelas, o BubbleSort apresentou tempos de execução de cerca de 10000 vezes os tempos das versões do quicksort e mergesort para vetores de 500000 elementos.

Já os algoritmos de complexidade linearítmica, como o MergeSort e o QuickSort, demonstraram ser escolhas sólidas para a maioria dos cenários, oferecendo um bom equilíbrio entre complexidade e desempenho, com uma boa gerência dos custos computacionais empregados.

Por fim, os algoritmos lineares, como o RadixSort e o BucketSort, apresentaram-se como os mais eficientes, especialmente para grandes volumes de dados. Porém, essa eficiência vem com alguns custos, como a grande quantidade de memória necessária para o BucketSort e a restrição a tipos específicos de dados para o RadixSort, uma vez que ele só pode ser aplicado a dados numéricos, e portanto, esses algoritmos não são adequados para todos os casos.

Sendo assim, dada a grande eficiência e baixos custos computacionais, algoritmos como o MergeSort e o QuickSort são amplamente utilizados em aplicações práticas, sendo frequentemente a escolha padrão em bibliotecas de ordenação em diversas linguagens de programação, e frequentemente são encontrados, sozinhos ou como combinações de dois ou mais algoritmos, em bibliotecas de ordenação usadas para aplicações que lidam com volumes gigantescos de dados.

Referências

[1] Knuth, D. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2009). *Introduction to Algorithms*. MIT Press.

[3] Ziviani, N. (2011). *Projeto de Algoritmos: com implementações em Pascal e C* (3ª ed.). Cengage Learning.

[4] Szwarcfiter, J. L., Markenzon, L. (2010). *Estruturas de Dados e seus Algoritmos* (3ª ed.). LTC.

5 Implementações dos Algoritmos

As implementações dos algoritmos de ordenação utilizados no estudo comparativo estão listadas a seguir

5.1 BubbleSort

:

```
// ============ BUBBLE SORT ============
void bolha(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
```

```
    est->trocas = 0;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {

            est->comparacoes++;

            if (vetor[j] > vetor[j + 1])
            {
                // Realiza a troca
                int aux = vetor[j];
                vetor[j] = vetor[j + 1];
                vetor[j + 1] = aux;

                est->trocas++;
            }
        }
    }
}
//
```

5.2 BubbleSort com Parada

:

```
void bolhaComParada(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
    int mudou = 1;

    while (mudou)
    {
        int j = 1;
        mudou = 0;
        int ultimo_indice_troca = 0;

        while (j < n)
        {
            est->comparacoes++;
            if (vetor[j - 1] > vetor[j])
            {
                int aux = vetor[j - 1];
                vetor[j - 1] = vetor[j];
                vetor[j] = aux;

                est->trocas++;
                mudou = 1;
                ultimo_indice_troca = j;
```

```
            }
            j = j + 1;
        }
        n = ultimo_indice_troca;
    }
}
//
```

5.3 InsertionSort

:

```
// ============ INSERCAO DIRETA ============
void ordenaDireta(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    for (int i = 1; i < n; i++)
    {
        int aux = vetor[i];
        int j = i - 1;

        while (j >= 0)
        {
            est->comparacoes++;
            if (aux < vetor[j])
            {
                vetor[j + 1] = vetor[j];
                est->trocas++;
                j = j - 1;
            }
            else
            {
                break;
            }
        }
        vetor[j + 1] = aux;
    }
}
//
```

5.4 BinarySort

:

```
// ============ INSERCAO BINARIA ============
void ordenaBinaria(int *v, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
```

```

    for (int i = 1; i < n; i++)
    {
        int aux = v[i];
        int esq = 0;
        int dir = i;

        while (esq < dir)
        {
            est->comparacoes++;
            int meio = (esq + dir) / 2;
            if (v[meio] <= aux)
            {
                esq = meio + 1;
            }
            else
            {
                dir = meio;
            }
        }

        for (int j = i; j > esq; j--)
        {
            v[j] = v[j - 1];
            est->trocas++;
        }
        v[esq] = aux;
    }
}

//
```

5.5 TernarySort

:

```

// ============ INSERCAO TERNARIA ============
void ordenaTernaria(int *v, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    for (int i = 1; i < n; i++)
    {
        int aux = v[i];
        int esq = 0;
        int dir = i - 1;

        while (esq <= dir)
        {
```

```
            int m1 = esq + (dir - esq) / 3;
            int m2 = dir - (dir - esq) / 3;

            est->comparacoes++;
            if (aux < v[m1])
            {
                dir = m1 - 1;
            }
            else
            {
                est->comparacoes++;
                if (aux > v[m2])
                {
                    esq = m2 + 1;
                }
                else
                {
                    esq = m1 + 1;
                    dir = m2 - 1;
                }
            }
        }

        for (int j = i; j > esq; j--)
        {
            v[j] = v[j - 1];
            est->trocas++;
        }
        v[esq] = aux;
    }
}

//
```

5.6 SelectionSort

:

```
// ============ SELECAO DIRETA ============
void selecaoDireta(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    for (int i = 0; i < n - 1; i++)
    {
        int menor = i;
        for (int j = i + 1; j < n; j++)
        {
            est->comparacoes++;
            if (vetor[j] < vetor[menor])
```

```
            {
                menor = j;
            }
        }

        if (i != menor)
        {
            int aux = vetor[i];
            vetor[i] = vetor[menor];
            vetor[menor] = aux;
            est->trocas++;
        }
    }
}

//
```

5.7 ShellSort

:

```

// ============ SHELL SORT ============
void shellSort(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    int gap = 1;
    while (gap < n)
    {
        gap = 3 * gap + 1;
    }

    while (gap >= 1)
    {
        for (int i = gap; i < n; i++)
        {
            int aux = vetor[i];
            int j = i - gap;

            while (j >= 0)
            {
                est->comparacoes++;
                if (aux < vetor[j])
                {
                    vetor[j + gap] = vetor[j];
                    est->trocas++;
                    j = j - gap;
                }
                else
```

```
                {
                    break;
                }
            }
            vetor[j + gap] = aux;
        }
        gap = gap / 3;
    }
}
//
```

5.8 MergeSort

:

```
// ============ MERGE SORT ============

void intercalar(int vetor[], int inicio, int meio, int final,
    Estatisticas *est)
{
    int i = inicio;
    int j = meio + 1;
    int k = 0;
    int tamanho = final - inicio + 1;
    int *temp = (int *)malloc(tamanho * sizeof(int));

    while (i <= meio && j <= final)
    {
        est->comparacoes++;
        if (vetor[i] <= vetor[j])
        {
            temp[k] = vetor[i];
            i++;
        }
        else
        {
            temp[k] = vetor[j];
            j++;
        }
        est->trocas++;
        k++;
    }

    while (i <= meio)
    {
        temp[k] = vetor[i];
        i++;
        k++;
        est->trocas++;
    }

```

```
    while (j <= final)
    {
        temp[k] = vetor[j];
        j++;
        k++;
        est->trocas++;
    }

    for (i = 0; i < tamanho; i++)
    {
        vetor[inicio + i] = temp[i];
        est->trocas++;
    }

    free(temp);
}

void mergeSort(int vetor[], int inicio, int fim, Estatisticas *est)
{
    if (inicio < fim)
    {
        int meio = (inicio + fim) / 2;
        mergeSort(vetor, inicio, meio, est);
        mergeSort(vetor, meio + 1, fim, est);
        intercalar(vetor, inicio, meio, fim, est);
    }
}

void mergeSortWrapper(int vetor[], int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
    if (n > 1)
    {
        mergeSort(vetor, 0, n - 1, est);
    }
}
//
```

5.9 HeapSort

:

```
// ============ HEAP SORT ============
void criaHeap(int *vetor, int inicio, int final, Estatisticas *est)
{
    int aux = vetor[inicio];
    int j = inicio * 2 + 1;

    while (j <= final)
    {
```

```
        if (j < final)
        {
            est->comparacoes++;
            if (vetor[j] < vetor[j + 1])
            {
                j = j + 1;
            }
        }

        est->comparacoes++;
        if (aux < vetor[j])
        {
            vetor[inicio] = vetor[j];
            est->trocas++;
            inicio = j;
            j = 2 * inicio + 1;
        }
        else
        {
            break;
        }
    }
    vetor[inicio] = aux;
}

void heapSort(int *vetor, int n, Estatisticas *est)

{
    est->comparacoes = 0;
    est->trocas = 0;

    for (int i = n / 2 - 1; i >= 0; i--)
    {
        criaHeap(vetor, i, n - 1, est);
    }

    for (int i = n - 1; i > 0; i--)
    {
        int aux = vetor[0];
        vetor[0] = vetor[i];
        vetor[i] = aux;
        est->trocas++;

        criaHeap(vetor, 0, i - 1, est);
    }
}
//
```

5.10 QuickSort (pivô no centro)

:

```c
// ============ QUICK SORT - PIVO CENTRO ============
int particaoCentro(int esq, int dir, int vetor[], Estatisticas *est
    )
{
    int i = esq;
    int j = dir;
    int pivo = vetor[(esq + dir) / 2];

    while (i <= j)
    {
        while (vetor[i] < pivo)
        {
            est->comparacoes++;
            i++;
        }
        est->comparacoes++;

        while (vetor[j] > pivo)
        {
            est->comparacoes++;
            j--;
        }
        est->comparacoes++;

        if (i <= j)
        {
            int aux = vetor[i];
            vetor[i] = vetor[j];
            vetor[j] = aux;
            est->trocas++;
            i++;
            j--;
        }
    }
    return i;
}

void quickSortRecursivoCentro(int vetor[], int esq, int dir,
    Estatisticas *est)
{
    while (esq < dir)
    {
        int p = particaoCentro(esq, dir, vetor, est);

        int tamanhoEsq = (p - 1) - esq;
        int tamanhoDir = dir - p;

        if (tamanhoEsq < tamanhoDir)
        {
            quickSortRecursivoCentro(vetor, esq, p - 1, est);
            esq = p;
```

```
        }
        else
        {
            quickSortRecursivoCentro(vetor, p, dir, est);
            dir = p - 1;
        }
    }
}

void quicksortcentro(int vetor[], int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
    quickSortRecursivoCentro(vetor, 0, n - 1, est);
}
//
```

5.11 QuickSort (pivô no fim)

:

```
// ============ QUICK SORT - PIVO FIM ============
int particaoFim(int esq, int dir, int vetor[], Estatisticas *est)
{
    int i = esq;
    int j = dir - 1;
    int pivo = vetor[dir];

    while (i <= j)
    {
        while (i <= j && vetor[i] < pivo)
        {
            est->comparacoes++;
            i++;
        }
        if (i <= j)
            est->comparacoes++;

        while (i <= j && vetor[j] > pivo)
        {
            est->comparacoes++;
            j--;
        }
        if (i <= j)
            est->comparacoes++;

        if (i <= j)
        {
            int aux = vetor[i];
            vetor[i] = vetor[j];
            vetor[j] = aux;
```

```
            est->trocas++;
            i++;
            j--;
        }
    }

    int aux = vetor[i];
    vetor[i] = vetor[dir];
    vetor[dir] = aux;
    est->trocas++;

    return i;
}

void quickSortRecursivoFim(int vetor[], int esq, int dir,
   Estatisticas *est)
{
    while (esq < dir)
    {
        int p = particaoFim(esq, dir, vetor, est);

        if (p - esq < dir - p)
        {
            quickSortRecursivoFim(vetor, esq, p - 1, est);
            esq = p + 1; // Loop trata o lado direito
        }
        else
        {
            quickSortRecursivoFim(vetor, p + 1, dir, est);
            dir = p - 1; // Loop trata o lado esquerdo
        }
    }
}

void quicksortfim(int vetor[], int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
    if (n > 0)
    {
        quickSortRecursivoFim(vetor, 0, n - 1, est);
    }
}

//
```

5.12 QuickSort (pivô na mediana)

:

```
// ============ QUICK SORT - MEDIANA ============
```

```c
int medianaDe3(int esq, int dir, int vetor[], Estatisticas *est)
{
    int meio = (esq + dir) / 2;

    est->comparacoes++;
    if (vetor[esq] > vetor[meio])
    {
        int aux = vetor[esq];
        vetor[esq] = vetor[meio];
        vetor[meio] = aux;
        est->trocas++;
    }

    est->comparacoes++;
    if (vetor[meio] > vetor[dir])
    {
        int aux = vetor[meio];
        vetor[meio] = vetor[dir];
        vetor[dir] = aux;
        est->trocas++;
    }

    est->comparacoes++;
    if (vetor[esq] > vetor[meio])
    {
        int aux = vetor[esq];
        vetor[esq] = vetor[meio];
        vetor[meio] = aux;
        est->trocas++;
    }

    return meio;
}

int particaoMediana(int esq, int dir, int vetor[], Estatisticas *
    est)
{
    int meio = medianaDe3(esq, dir, vetor, est);
    int pivo = vetor[meio];
    int i = esq;
    int j = dir;

    while (i <= j)
    {
        while (vetor[i] < pivo)
        {
            est->comparacoes++;
            i++;
        }
        est->comparacoes++;

```

```
        while (vetor[j] > pivo)
        {
            est->comparacoes++;
            j--;
        }
        est->comparacoes++;

        if (i <= j)
        {
            int aux = vetor[i];
            vetor[i] = vetor[j];
            vetor[j] = aux;
            est->trocas++;
            i++;
            j--;
        }
    }
    return i;
}

void quickSortRecursivoMediana(int vetor[], int esq, int dir,
    Estatisticas *est)
{
    while (esq < dir)
    {
        int p = particaoMediana(esq, dir, vetor, est);

        // Calculamos os tamanhos
        int tamanhoEsq = (p - 1) - esq;
        int tamanhoDir = dir - p;

        if (tamanhoEsq < tamanhoDir)
        {
            quickSortRecursivoMediana(vetor, esq, p - 1, est);
            esq = p;
        }
        else
        {
            quickSortRecursivoMediana(vetor, p, dir, est);
            dir = p - 1;
        }
    }
}

void quicksortmediana(int vetor[], int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;
    if (n > 0)
    {
        quickSortRecursivoMediana(vetor, 0, n - 1, est);
```

```
    }
}
//
```

5.13 RadixSort

:

```
// ============ RADIX SORT ============
void radixSort(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    // Encontrar o maior n mero para saber o n mero de d gitos
    int maior = vetor[0];
    for (int i = 1; i < n; i++)
    {
        est->comparacoes++;
        if (vetor[i] > maior)
            maior = vetor[i];
    }

    // Aloca  o din mica para evitar Stack Overflow com 500k
       elementos
    int *output = (int *)malloc(n * sizeof(int));
    if (output == NULL)
        return; // Tratamento b sico de erro

    for (int exp = 1; maior / exp > 0; exp *= 10)
    {
        int count[10] = {0};

        // Contagem de ocorr ncias (frequ ncia)
        for (int i = 0; i < n; i++)
        {
            count[(vetor[i] / exp) % 10]++;
        }

        // Posi  o acumulada (Prefix Sum)
        for (int i = 1; i < 10; i++)
        {
            count[i] += count[i - 1];
        }

        for (int i = n - 1; i >= 0; i--)
        {
            output[count[(vetor[i] / exp) % 10] - 1] = vetor[i];
            count[(vetor[i] / exp) % 10]--;
            est->trocas++; // Movimenta  o para o vetor auxiliar
        }
```

```

        // Copiando de volta para o vetor original
        for (int i = 0; i < n; i++)
        {
            vetor[i] = output[i];
            est->trocas++; // Movimenta  o de volta para o vetor principal
        }
    }

    free(output); // Libera a mem ria alocada
}
//
```

5.14 BucketSort

:

```
// ============ BUCKET SORT ============
void bucketSort(int *vetor, int n, Estatisticas *est)
{
    est->comparacoes = 0;
    est->trocas = 0;

    int maior = vetor[0];
    for (int i = 1; i < n; i++)
    {
        est->comparacoes++;
        if (vetor[i] > maior)
        {
            maior = vetor[i];
        }
    }

    int *buckets = (int *)calloc(maior + 1, sizeof(int));

    // Verifica  o de seguran a (caso falte mem ria RAM)
    if (buckets == NULL)
        return;

    for (int i = 0; i < n; i++)
    {
        buckets[vetor[i]]++;
    }
    int k = 0;
    for (int i = 0; i <= maior; i++)
    {
        // Se o balde i tem elementos, coloca eles de volta no vetor original
        for (int j = 0; j < buckets[i]; j++)
        {
```

```
            vetor[k++] = i;
            est->trocas++;
        }
    }

    free(buckets); // Libera a mem ria alocada
}

//
```